

— THE BOLIDE PROGRAMMING LANGUAGE

Bolide

从入门到精通

基于 Cranelift 的 JIT / AOT 编译型语言 · 系统教程

原生性能

Python 风格语法

ARC 内存模型

并发 / async

FFI

AOT 发布

version 0.12.3 · zero → mastery

关于本书

书名	Bolide 从入门到精通
副标题	从第一个程序到 AOT 原生发布的完整学习路径
版本基准	Bolide 0.12.3
编译后端	Cranelfit (JIT 与 AOT 双模式)
文件扩展名	.bl
覆盖主题	语法 · 类型系统 · 内存模型 · 并发 · FFI · 标准库 · 工程实践
适合读者	第一次接触 Bolide 的开发者，以及希望系统掌握其工程能力的进阶读者
项目地址	github.com/streetartist/bolide

本书不是 API 清单，而是一条学习路径：从写出第一个程序开始，逐步走向类型系统、内存模型、并发、AOT 发布和与 C 互操作。

建议阅读顺序：第 1-4 章掌握安装、运行、基础语法和控制流；第 5-9 章掌握函数、一等函数、闭包、集合、类、模块和错误处理；第 10-13 章理解类型系统、内存管理、并发与异步；第 14-18 章学习 FFI、AOT 发布、Web/GUI 标准库、调试诊断、测试与性能优化；第 19-20 章完成综合项目并查阅语法速查。

目录 Contents

01 认识 Bolide	4
02 安装、运行与项目结构	6
03 第一个程序	8
04 基础语法：变量、类型、表达式	10

05	控制流、作用域与求值顺序	13
06	函数、参数与泛型	16
07	一等函数、闭包与高阶编程	20
08	字符串、列表、字典、元组与切片	23
09	类、对象与面向对象	27
10	枚举、模式匹配与错误处理	30
11	模块系统与包管理器	34
12	类型系统与类型推断	37
13	内存管理：ARC、生命周期、weak、unowned	39
14	并发、线程、通道与 async/await	42
15	FFI 与 AOT 发布	46
16	Web、模板、数据库与 GUI 标准库	49
17	报错诊断、调试与测试	54
18	性能优化与工程实践	56
19	综合项目：命令行任务管理器	58
20	附录：常用语法速查	61
—	结语	65

CHAPTER 01

认识 Bolide

Bolide 的核心目标是把脚本语言的表达效率和原生编译语言的部署体验结合起来。

学习目标

- 理解 Bolide 的定位：高层表达力 + 原生性能
- 区分 JIT 与 AOT 两种运行方式及其适用阶段
- 能写出并运行一个最小程序

它有两种运行方式：

- JIT: `bolide run main.bl`，适合开发、调试、快速试验。
- AOT: `bolide compile main.bl -o main`，适合发布独立可执行文件。



图 1-1 同一份源码经 Craneflight 后端走向 JIT 或 AOT 两条路径

Bolide 适合这些场景：

- 写性能足够好的命令行工具。
- 写需要原生部署的服务端程序。
- 写需要 C 互操作的系统边界代码。
- 写带 GUI 的小工具。
- 用高级语法组织中大型业务逻辑。

Bolide 文件通常使用 `.bl` 扩展名。

1.1 它在语言光谱中的位置

把 Bolide 和两类常见语言放在一起，能更快建立直觉：它借用了脚本语言的书写方式，却走原生编译的部署路线。

表 1-1 Bolide 与常见语言的定位对照（基于本书所述特性）

维度	Python 风格脚本	系统级编译语言	Bolide
书写风格	简洁、高层	偏底层、显式	简洁、高层（切片、具名参数、闭包）
运行模型	解释执行	原生编译	JIT 开发 + AOT 发布
类型系统	多为动态	静态	静态 + 类型推断，保留 <code>dynamic</code>
内存管理	GC	手动 / 所有权	ARC + 所有权 / 生命周期注解
部署形态	需运行时	独立可执行	AOT 单文件可执行

1.2 第一个最小程序

一个最小程序：

```
print("hello Bolide");
```

BOLIDE

JIT 运行：

```
bolide run hello.bl
```

BASH

AOT 编译：

```
bolide compile hello.bl -o hello
```

BASH

本章小结

- Bolide 用一套语法、两种运行模式覆盖「快速开发」与「原生发布」。
- 开发期用 `bolide run`（JIT），发布期用 `bolide compile`（AOT）。
- 它的设计取舍是：尽量保留高层表达力，同时不放弃原生性能与部署体验。

CHAPTER 02

安装、运行与项目结构

学习目标

- 从源码构建 Bolide，并运行示例
- 区分单文件项目与包结构项目
- 掌握 `run` / `compile` / `new` / `add` / `install` 等常用命令

从源码构建：

```
git clone https://github.com/streetartist/bolide.git
cd bolide
cargo build --release
```

BASH

运行示例：

```
cargo run --release -- run examples/hello.bl
```

BASH

发布包通常提供 `bolide` 或 `bolide.exe`。下载后可以直接运行：

```
bolide run your_program.bl
```

BASH

一个简单项目可以只有一个文件：

```
hello.bl
```

TEXT

稍复杂的项目建议使用包结构：

```
my_app/
  bolide.toml
  src/
    main.bl
    util.bl
```

TEXT

`bolide.toml` 描述包名和依赖。`bolide run` 和 `bolide compile` 会自动向上查找 `bolide.toml`，解析依赖并注入编译器。

2.1 常用命令

```
bolide run src/main.bl
bolide compile src/main.bl -o app
bolide compile src/lib.bl --lib --header
bolide new my_app
bolide add dep-name@1.0.0
bolide install
```

BASH

表 2-1 命令速查

命令	作用	典型阶段
<code>bolide run</code>	JIT 直接运行源码	开发 / 调试
<code>bolide compile -o</code>	AOT 编译为可执行文件	发布
<code>bolide compile --lib --header</code>	编译静态库并生成 C 头文件	对外提供库
<code>bolide new</code>	创建包结构项目	项目初始化
<code>bolide add</code> / <code>install</code>	添加 / 安装依赖	依赖管理

本章小结

- 小脚本一个 `.bl` 即可；中型以上用 `bolide.toml` + `src/` 包结构。
- 构建工具会自动向上查找 `bolide.toml` 并解析依赖。

CHAPTER 03

第一个程序

学习目标

- 用 `let` 定义变量，理解类型注解可省略的场景
- 读取用户输入并做类型转换

创建 `hello.bl`：

```
let name: str = "Bolide";  
print("hello " + name);
```

BOLIDE

运行：

```
bolide run hello.bl
```

BASH

Bolide 使用 `let` 定义变量。类型注解可以写，也可以在很多场景下省略：

```
let a: int = 10;  
let b = 20;  
print(a + b);
```

BOLIDE

读取用户输入，可带提示或不带：

```
let name: str = input("name: "); // 带提示  
let content: str = input();      // 无提示  
print("hello " + name);
```

BOLIDE

`input()` 返回字符串。需要数字时显式转换：

```
let raw: str = input("age: ");  
let age: int = int(raw);  
print(age + 1);
```

BOLIDE

常见错误

`input()` 永远返回 `str`。直接对它做算术会出错——务必先用 `int(...)` 或 `float(...)` 转换。

本章小结

- `let` 绑定变量，类型注解在能推断时可省略。
- 边界数据（如输入）要在进入业务逻辑前显式转换类型。

CHAPTER 04

基础语法：变量、类型、表达式

学习目标

- 掌握变量绑定与赋值（含复合赋值）
- 认识内建基本类型与字面量后缀
- 理解运算符优先级与短路求值

4.1 变量

```
let x: int = 42;
let pi: float = 3.14159;
let ok: bool = true;
let name: str = "Ada";
```

BOLIDE

Bolide 的变量绑定使用 `let`。后续赋值使用 `=`：

```
let n: int = 1;
n = n + 1;
n += 10;
```

BOLIDE

4.2 基本类型

常用内建类型：

表 4-1 内建基本类型

类型	含义
<code>int</code>	64 位整数
<code>float</code>	双精度浮点
<code>bool</code>	布尔值
<code>str</code>	字符串
<code>bigint</code>	任意精度整数
<code>decimal</code>	十进制定点 / 高精度数
<code>dynamic</code>	动态值
<code>none</code>	空值字面量

示例：

```
let big: bigint = 123456789012345678901234567890b;
let money: decimal = 19.99d;
let empty = none;
```

BOLIDE

提示

字面量后缀 `b` 表示 `bigint`、`d` 表示 `decimal`，大小写均可（`999b` 与 `999B` 等价）。涉及金额、精确计算时用 `decimal` 而非 `float`，可避免二进制浮点误差。

4.3 数字字面量与字符串转义

数字字面量支持下划线分隔，便于阅读大数：

```
let million: int = 1_000_000;
```

BOLIDE

字符串支持转义序列 `" \ \n \t \r \0`：

```
let quoted: str = "he said \"hi\" \nsecond line";
```

BOLIDE

4.4 表达式与运算符

```
let a = 10 + 3 * 2;
let b = (10 + 3) * 2;
let c = a > b;
let d = true and not false;
let e = false or expensive_check();
```

BOLIDE

`and` / `or` 是短路求值：右侧只在需要时执行。

4.5 类型转换

Bolide 提供完整的类型转换函数。`int()` 对浮点是截断（非四舍五入）：

BOLIDE

```
let a: int = int(3.7);           // float → int (截断) = 3
let b: int = int("123");         // str → int = 123
let c: int = int(999B);          // bigint → int
let d: int = int(45.6D);         // decimal → int = 45

let e: float = float(100);       // int → float = 100.0
let f: float = float("2.718");  // str → float

let h: str = str(12345);         // int → str
let j: str = str(true);         // bool → str = "true"

let m: bigint = bigint(100);     // int → bigint
let n: decimal = decimal(3.14); // float → decimal
```

表 4-2 类型转换函数一览

函数	接受	说明
<code>int(x)</code>	float / str / bigint / decimal	浮点截断取整
<code>float(x)</code>	int / str / decimal	转双精度浮点
<code>str(x)</code>	任意基本类型	转字符串表示
<code>bigint(x)</code> / <code>decimal(x)</code>	数值 / 字符串	转高精度类型

本章小结

- `let` 绑定 + `=` / `+=` 赋值；类型注解按需书写。
- 字面量后缀 `b` / `d`（大小写均可）区分 `bigint` / `decimal`；数字可用 `_` 分隔；字符串支持 `\n` `\t` `\"` `\\` 等转义。
- `and` / `or` 短路求值；`int()` 对浮点截断而非四舍五入。

CHAPTER 05

控制流、作用域与求值顺序

学习目标

- 使用 `if/elif/else`、`while`、`for` 组织控制流
- 用 `break/continue` 精细控制循环
- 理解块级作用域与参数求值顺序

5.1 if / elif / else

```
let n = int(input("n: "));

if n > 0 {
    print("positive");
} elif n < 0 {
    print("negative");
} else {
    print("zero");
}
```

BOLIDE

5.2 while

```
let i = 0;
while i < 5 {
    print(i);
    i += 1;
}
```

BOLIDE

5.3 for 与 range

`range` 支持单参、起止、步长，步长可为负：

```
for i in range(5) { print(i); }           // 0 1 2 3 4
for i in range(3, 7) { print(i); }        // 3 4 5 6
for i in range(0, 10, 2) { print(i); }    // 0 2 4 6 8
for i in range(10, 0, -2) { print(i); }   // 10 8 6 4 2 (负步长)
```

BOLIDE

遍历列表：

```
let nums: list<int> = [10, 20, 30];
for n in nums {
    print(n);
}
```

BOLIDE

Python 风格的字典遍历，一次解构出键和值：

```
let scores = {"Alice": 100, "Bob": 85};
for k, v in scores {
    print(k); // 键
    print(v); // 值
}
```

BOLIDE

5.4 复合赋值与 break / continue

完整的复合赋值运算符：

```
let n: int = 10;
n += 5; // 15
n -= 3; // 12
n *= 2; // 24
n /= 4; // 6
n %= 4; // 2
```

BOLIDE

```
for i in range(10) {
    if i = 3 {
        continue;
    }
    if i = 8 {
        break;
    }
    print(i);
}
```

BOLIDE

5.5 作用域

块会创建局部作用域。内层变量不会污染外层：

```

let x = 1;

if true {
    let x = 2;
    print(x); // 2
}

print(x); // 1

```

BOLIDE

5.6 求值顺序

函数参数、具名参数、变长参数和复合表达式按源码顺序求值。依赖副作用时，应把复杂表达式拆成临时变量，让顺序更清晰：

```

let a = next();
let b = next();
print(pair(a, b));

```

BOLIDE

提示

当表达式里有副作用（如读取流、递增计数器）时，拆成命名的临时变量，既保证顺序，又让代码意图一目了然。

本章小结

- 控制流关键字与主流语言一致，块用 `{ }` 包裹。
- `range` 支持起止与负步长；字典用 `for k, v in d` 同时取键值。
- 复合赋值含 `+= -= *= /= %=`；每个块是独立作用域。

CHAPTER 06

函数、参数与泛型

学习目标

- 定义函数与返回类型
- 使用默认参数、具名参数、变长参数
- 理解参数模式（借用 / owned / ref）与泛型函数

6.1 定义函数

```
fn add(a: int, b: int) → int {  
    return a + b;  
}  
  
print(add(1, 2));
```

BOLIDE

无返回值函数可以省略返回类型：

```
fn greet(name: str) {  
    print("hello " + name);  
}
```

BOLIDE

6.2 默认参数

```
fn greet(name: str = "world", punctuation: str = "!") {  
    print("hello " + name + punctuation);  
}  
  
greet();  
greet("Bolide");
```

BOLIDE

6.3 具名参数

```
greet(name="Ada", punctuation=".");  
greet(punctuation="?", name="Bob");
```

BOLIDE

Bolide 同时接受 `name=value` 和 `name: value` 风格的具名实参。

6.4 变长参数

```
fn total(base: int = 0, *nums: int) → int {
    let sum = base;
    for n in nums {
        sum += n;
    }
    return sum;
}

print(total(10, 1, 2, 3));
```

BOLIDE

关键字变长参数，函数体中类型为 `dict<str, T>`：

```
fn total(base: int = 10, *nums: int, **opts: int) → int {
    let sum: int = base;
    for n in nums {
        sum += n;
    }
    if opts.contains("bonus") {
        sum += opts["bonus"];
    }
    return sum;
}
```

BOLIDE

调用侧可以用 `*list_expr` 展开列表、`**dict_expr` 展开字典；未匹配的具名实参会落入 `**opts`：

```
let xs: List<int> = [2, 3];
let kwargs: dict<str, int> = {"bonus": 4};

print(total()); // 10
print(total(1, *xs, **kwargs)); // 展开列表与字典
print(total(base=5, bonus=7)); // 未知具名参数进入 **opts
```

BOLIDE

参数顺序约束

`*args` 必须位于普通参数之后，`**kwargs` 必须是最后一个参数。当函数没有 `**kwargs` 接收时，传入未知具名参数会编译报错。

6.5 参数模式

Bolide 支持普通借用、所有权传递和引用修改：

```
fn take(owned value: list<int>) {
    print(value);
}

fn bump(ref n: int) {
    n += 1;
}
```

表 6-1 三种参数模式

模式	语义	典型用途
默认（借用）	只读访问，不夺走所有权	大多数函数
owned	把值的所有权交给函数	消费、转移资源
ref	允许函数修改调用方变量	原地更新

6.6 泛型函数

函数名后可声明 `<T>` 或 `<T, U>` 类型参数。调用点通常无需写类型实参，编译器按实参推断，并在后端编译前单态化为具体实例：

```
fn id<T>(x: T) → T {
    return x;
}

fn pair<T, U>(a: T, b: U) → (T, U) {
    return (a, b);
}

fn wrap<T>(x: T) → list<T> {
    return [x];
}

print(id(42));           // 42
print(id("hello"));      // hello
print(pair(10, "x"));     // (10, "x")
print(wrap(7));          // [7]
```

当前限制

泛型支持顶层函数的直接调用与嵌套调用；泛型方法、以及把未实例化的泛型函数作为一等值传递暂未支持。

本章小结

- 参数能力完整：默认值、具名、`*args`（→ `list`）/ `**kwargs`（→ `dict`），调用侧可 `*` / `**` 展开。
- `owned` 转移所有权、`ref` 原地修改，默认是借用。
- 泛型用 `<T>` / `<T, U>`，调用点单态化；泛型方法暂不支持。

CHAPTER 07

一等函数、闭包与高阶编程

学习目标

- 把函数当作值传递、返回、存储
- 用闭包字面量捕获局部变量
- 使用 `map` / `filter` 表达数据变换

函数是值：可以赋给变量、作为参数、作为返回值、放进集合。

命名约定

`fn` 只用于函数声明和闭包字面量；`func(T...) → R` 是唯一的用户级函数值类型。C 函数指针、`*void` 等属于 FFI/编译器内部细节，不应暴露成普通业务类型。

```
fn add1(x: int) → int { return x + 1; }
fn double(x: int) → int { return x * 2; }

let f = add1;
print(f(10)); // 11

let g: func(int) → int = double;
print(g(10)); // 20
```

BOLIDE

7.1 函数作为参数

```
fn apply(callback: func(int) → int, x: int) → int {
    return callback(x);
}

print(apply(double, 21)); // 42
```

BOLIDE

7.2 返回函数

```
fn pick(which: int) → func(int) → int {
    if which == 0 {
        return add1;
    }
    return double;
}

let f = pick(1);
print(f(7)); // 14

// 返回值可直接链式调用
print(pick(0)(7)); // 8
print(pick(1)(7)); // 14

// 函数存入列表，按下标取出再调用
let fns: list<func(int) → int> = [add1, double];
print(fns[0](5)); // 6
print(fns[1](5)); // 10
```

BOLIDE

7.3 闭包

闭包使用 `fn(...) → T { ... }` 字面量，会自动捕获局部变量：

```
fn make_adder(delta: int) → func(int) → int {
    return fn(x: int) → int {
        return x + delta;
    };
}

let add10 = make_adder(10);
print(add10(5)); // 15
```

BOLIDE

闭包适合延迟执行、回调、组合逻辑。

7.4 map / filter

```
fn double(x: int) → int { return x * 2; }
fn is_even(x: int) → bool { return x % 2 == 0; }

let nums: list<int> = [1, 2, 3, 4];
print(nums.map(double)); // [2, 4, 6, 8]
print(nums.filter(is_even)); // [2, 4]
```

BOLIDE

本章小结

- 函数是一等值，类型写作 `func(参数) → 返回`。
- 闭包字面量自动捕获环境，适合回调与逻辑组合。
- `map` / `filter` 让数据变换更声明式。

CHAPTER 08

字符串、列表、字典、元组与切片

学习目标

- 掌握字符串常用方法与拼接
- 使用列表、字典、元组组织数据
- 用 Python 风格切片截取序列

8.1 字符串

`str` 提供丰富的方法，调用风格类似 Python：

```
let s: str = "Hello, World";

print(s.len());           // 12
print(s.upper());         // HELLO, WORLD
print(s.lower());         // hello, world
print(s.contains("World")); // 1
print(s.find("World"));   // 7
print(s.starts_with("Hell")); // 1
print(s.ends_with("rld")); // 1
print(s.replace("l", "L")); // HeLLo, WorLd
print(s.count("l"));       // 3

print(" trim me ".trim()); // trim me
print("ab".repeat(3));     // ababab
print(s.substring(0, 5));  // Hello
print(s.char_at(1));       // e

let parts: list<str> = "a,b,c".split(",");
print(parts);           // ["a", "b", "c"]
```

常用别名：`length()` / `size()`、`strip()`、`index_of()`、`includes()`、`substr()`。字符串可直接用 `+` 拼接。

Unicode 注意

字符串索引和切片按 **Unicode 码点** 处理；但 `len()` 当前返回 **UTF-8 字节长度**。处理多字节字符时需留意这一区别。

8.2 列表

列表提供成套的 Python 风格操作：

```
let nums: list<int> = [3, 1, 4, 1, 5, 9];

nums.push(10);           // 追加
let x: int = nums.pop();  // 弹出末尾
print(nums.len());

print(nums[0]);           // 取元素
nums[0] = 100;            // 设元素

nums.insert(1, 42);       // 索引 1 处插入
let removed: int = nums.remove(2); // 移除索引 2

print(nums.contains(4));  // 是否包含 (0/1)
print(nums.index_of(4));  // 索引, 找不到返回 -1
print(nums.count(1));     // 出现次数

print(nums.first());      // 第一个
print(nums.last());       // 最后一个
print(nums.is_empty());   // 是否为空

nums.reverse();           // 原地反转
nums.sort();              // 原地排序

let more: list<int> = [100, 200];
nums.extend(more);        // 扩展
let copy: list<int> = nums.copy(); // 复制
nums.clear();             // 清空
```

BOLIDE

8.3 字典

```
// 强类型字典
let scores: dict<str, int> = {"Alice": 100, "Bob": 90};
print(scores["Alice"]); // 100

scores["Charlie"] = 95; // 插入/更新
scores.remove("Bob");   // 删除
print(scores.len());
print(scores.contains("Alice"));
print(scores.keys());   // 所有键
print(scores.values()); // 所有值
```

BOLIDE

键值异构时会自动推导为 `dict<dynamic, dynamic>` 并装箱：


```
let profile = {"name": "Bolide", 1: "Version", "active": true};
print(profile["name"]); // "Bolide"
print(profile[1]);      // "Version"
```

BOLIDE

8.4 元组

```
let point: (int, int) = (3, 4);
print(point[0]);
print(point[1]);
```

BOLIDE

8.5 切片

字符串、列表、元组都支持 `seq[start:end:step]`；三者均可省略，并支持负索引与负步长：

```
let text: str = "Hello, World";
print(text[0:5]); // Hello
print(text[7:]);  // World
print(text[::2]); // Hlo ol
print(text[::-1]); // dlroW ,olleH (反转)
print(text[-1]);  // d (负索引)

let nums: list<int> = [10, 20, 30, 40, 50];
print(nums[1:4]);   // [20, 30, 40]
print(nums[-2:]);   // [40, 50]
print(nums[::-1]);  // [50, 40, 30, 20, 10]

// 元组切片返回元组
let t: (int, int, int, int) = (1, 2, 3, 4);
let mid: (int, int) = t[1:3];
print(mid);          // (2, 3)
```

BOLIDE

表 8-1 切片语法速记

写法	含义
<code>seq[1:4]</code>	索引 1 到 3（左闭右开）
<code>seq[:3]</code> / <code>seq[2:]</code>	到索引 2 / 从索引 2 起
<code>seq[::2]</code>	步长为 2
<code>seq[-1]</code> / <code>seq[-2:]</code>	负索引从末尾计
<code>seq[::-1]</code>	反向（步长 -1）

本章小结

- 字符串方法丰富，注意「索引按码点、`len()` 按字节」。
- 列表方法成套：`push/pop/insert/remove/sort/reverse/extend/copy` 等。
- 字典支持强类型与混合类型；切片对三种序列统一适用，支持负索引/负步长。

CHAPTER 09

类、对象与面向对象

学习目标

- 定义类、字段与方法
- 用默认值或构造参数初始化对象
- 通过继承复用与覆写行为

类用于组织数据和行为。方法不需要声明 `self` 形参，在方法体内直接用 `self` 访问字段：

```
class Point {  
    x: int;  
    y: int;  
  
    fn distance() → int {  
        return self.x * self.x + self.y * self.y;  
    }  
  
    fn move_by(dx: int, dy: int) {  
        self.x = self.x + dx;  
        self.y = self.y + dy;  
    }  
}  
  
// 用构造函数按字段顺序初始化  
let p: Point = Point(3, 4);  
print(p.distance()); // 25  
  
p.move_by(1, 1);  
print(p.x); // 4  
print(p.y); // 5
```

字段也可以带默认值：

```

class Counter {
    value: int = 0;

    fn inc() {
        self.value = self.value + 1;
    }

    fn get() → int {
        return self.value;
    }
}

let c = Counter();
c.inc();
c.inc();
print(c.get()); // 2

```

9.1 继承

子类用 `class Child: Parent` 声明，继承父类字段与方法；构造时按「父类字段 + 子类字段」的顺序传参：

```

class Animal {
    age: int;
    fn get_age() → int { return self.age; }
}

class Dog: Animal {
    name: int;
    fn bark() → int { return 100; }
}

let dog: Dog = Dog(3, 42); // age=3, name=42
print(dog.get_age());      // 3 (继承的方法)
print(dog.bark());         // 100

```

设计建议

对象适合表达拥有状态的实体；纯计算优先用函数。不要为没有状态的逻辑强行包一层类。

本章小结

- 类聚合字段与方法，方法不写 `self` 形参，体内用 `self.field` 访问。
- 字段可带默认值；构造函数按字段声明顺序接收实参。
- `class Child: Parent` 继承字段与方法，构造按父字段在前的顺序传参。

CHAPTER 10

枚举、模式匹配与错误处理

学习目标

- 用 `enum` / `union` 建模代数数据类型
- 用 `match` 做模式匹配
- 用 `try/catch/finally` 与自定义错误类处理异常

10.1 枚举与 union

Bolide 支持代数数据类型风格的 `enum` / `union` :

```
enum OptionInt {  
    Some(int),  
    None,  
}
```

BOLIDE

模式匹配:

```
fn describe(x: OptionInt) → str {  
    match x {  
        OptionInt.Some(v) ⇒ {  
            return "value=" + str(v);  
        },  
        OptionInt.None() ⇒ {  
            return "none";  
        },  
    }  
}
```

BOLIDE

10.2 throw / try / catch / finally

异常用于非预期、需要跨层中止的错误。`throw` 只能抛出 `Error` 或 `Error` 子类, `catch` 也只能按 `Error` 体系捕获。

```
try {
    throw Error("boom");
} catch (e: Error) {
    print("caught: " + e.message);
} finally {
    print("cleanup");
}
```

BOLIDE

finally 无论是否抛错都会执行，适合释放资源、关闭连接、记录日志。

可选的 **throws** 注解用于说明函数可能抛出的异常：

```
fn load_config(path: str) throws Error → str {
    throw Error("missing config: " + path);
}
```

BOLIDE

10.3 内置 Error 类与自定义错误

编译器内置一个 **Error** 类（单字段 **message: str**），无需声明即可直接使用。自定义子类继承 **Error** 后，可被基类 **catch** 统一捕获（子类匹配）：

```
// 直接抛出/捕获内置 Error
try {
    throw Error("boom");
} catch (e: Error) {
    print(e.message); // boom
}

// 自定义子类，被基类 catch 捕获
class MyError: Error {}

try {
    throw MyError("custom failure");
} catch (e: Error) {
    print(e.message); // custom failure
}
```

BOLIDE

throw 不再接受任意值；基础类型如 **int**、**str** 不能直接抛出或捕获。异常会跨函数传播到最近的匹配 **catch**，**finally** 在展开过程中仍会执行。

10.4 跨函数传播与重新抛出

```
class ParseError: Error {}

fn parse_config() {
    throw ParseError("bad syntax");
}

try {
    parse_config();
} catch (e: ParseError) {
    print("parse failed: " + e.message);
} catch (e: Error) {
    print("other error: " + e.message);
}
```

BOLIDE

10.5 Option / Result 与 ?

值可能缺失时用 `Option<T>`，可恢复错误用 `Result<T, E>`：

```
fn parse_id(raw: str) → Result<int, Error> {
    if raw.len() == 0 {
        return Result.Err(Error("empty id"));
    }
    return Result.Ok(int(raw));
}

fn load_id(raw: str) → Result<int, Error> {
    let id: int = parse_id(raw)?;
    return Result.Ok(id + 1);
}
```

BOLIDE

`expr?` 会解包 `Result.Ok(v)` / `Option.Some(v)`；遇到 `Result.Err(e)` / `Option.None()` 时从当前函数早返回。

10.6 Result 与异常互转

```
fn init() {
    let id: int = parse_id("42")!;
    print(id);
}

let result: Result<int, Error> = try {
    let id: int = parse_id("42")!;
    id + 10;
};
```

BOLIDE

`expr!` 把 `Result.Err(e)` 升级为异常；`try { ... }` 表达式把块内异常捕获成 `Result<T, Error>`。

实践建议

- 缺失值用 `Option<T>`，可恢复错误用 `Result<T, E>`。
- 跨层异常使用明确的错误类，并继承内置 `Error`。
- 只有非预期错误才用异常，不把 `catch` 当普通业务分支。
- `finally` 只做清理，不写复杂业务逻辑。

本章小结

- `enum` / `union` + `match` 表达「多种形态之一」并强制穷尽处理。
- 内置 `Error`（含 `message`）开箱即用，子类可被基类 `catch` 捕获；异常支持跨函数传播。
- `Result` / `Option` 负责可恢复路径，`?`、`!`、`try` 表达式负责低成本互转。
- `finally` 保证清理。

CHAPTER 11

模块系统与包管理器

学习目标

- 用 `import` 导入文件、模块路径与别名
- 初始化包、添加依赖、安装依赖
- 建立清晰的工程目录约定

导入本地文件：

```
import "math_utils.bl";
```

BOLIDE

导入模块路径：

```
import math.utils;
```

BOLIDE

别名：

```
import "long/path/to/mod.bl" as mod;
```

BOLIDE

包结构：

```
my_app/  
  bolide.toml  
  src/  
    main.bl
```

TEXT

创建项目：

```
bolide new my_app
```

BASH

添加依赖：

```
bolide add name@1.0.0  
bolide add https://github.com/org/pkg.git --tag v1.0.0  
bolide add ../local_pkg --path
```

BASH

安装依赖：

```
bolide install
```

BASH

11.1 bolide.toml 与依赖来源

依赖可来自 **git 仓库**、**本地路径**或 **registry 索引**三种：

```
[package]
name = "myapp"           # 必填，作为依赖被引用时的命名空间
version = "0.1.0"        # 必填
lib = "src/lib.bl"       # 可选，包入口（默认 src/lib.bl）

[dependencies]
http = { git = "https://github.com/bolide-lang/http.git", ref = "v1.2.0" }
utils = { path = "../utils" }
db = { version = "0.3.0", registry = "https://registry.bolide.dev" }
```

bolide install 会解析依赖并生成 **bolide.lock**。缓存位于 **%LOCALAPPDATA%\bolide**（Windows）或 **~/.cache/bolide**（Linux/macOS），可用 **BOLIDE_CACHE_DIR** 覆盖。依赖以包名作为命名空间使用：

```
import utils;           // 解析到 utils 包入口
import utils as u;      // 别名
import "utils/extra.bl"; // 导入包内其他源文件

print(utils.greet());
```

11.2 路径解析与作用域

导入路径按确定性顺序解析（不依赖进程工作目录）：绝对路径原样使用 → 相对路径基于**导入方源文件目录** → 包管理器依赖 → **BOLIDE_HOME** → 可执行文件所在目录。

关于变量作用域，需要记住一条全局/局部规则：

表 11-1 全局与局部变量

声明位置	性质	说明
顶层 let	全局变量	函数内可读取与赋值（GUI 回调依赖这一点）
函数内 let	局部变量	同名时遮蔽全局变量

全局与局部能力一致：都可作 **ref** 实参、可作通道用于 **send** / **recv** 收发与 **select**、支持全部类型。

工程建议

- 一个文件只放一类概念。
- 公共函数放在 `src/lib.bl` 或 `src/*.bl`。
- 示例程序放 `examples/`，回归测试放 `tests/`。

本章小结

- `import` 支持文件、模块路径与 `as` 别名；符号位于以文件名命名的命名空间，不污染全局。
- 依赖来自版本号 / Git / 本地路径，`install` 生成 `bolide.lock`。
- 顶层 `let` 是全局变量，函数内可读写；同名时局部遮蔽全局。

CHAPTER 12

类型系统与类型推断

学习目标

- 理解「静态类型 + 类型推断」的取舍
- 知道何时该显式标注复杂边界类型
- 恰当（而非滥用）地使用 `dynamic`

Bolide 是静态类型语言，但很多地方可以推断类型。

```
let x = 1;           // int
let s = "hello";     // str
let xs = [1, 2];     // list<int>
```

BOLIDE

复杂边界建议显式标注：

```
let callbacks: list<func(int) → int> = [add1, double];
let table: dict<str, list<int>> = {"a": [1, 2]};
```

BOLIDE

函数签名建议明确写出：

```
fn score(name: str, values: list<int>) → int {
  let total = 0;
  for v in values {
    total += v;
  }
  return total;
}
```

BOLIDE

12.1 类型一览

表 12-1 Bolide 类型系统

类型	说明	示例
<code>int</code>	64 位整数	<code>let x: int = 42;</code>
<code>float</code>	64 位浮点	<code>let pi: float = 3.14;</code>
<code>bool</code>	布尔值	<code>let f: bool = true;</code>
<code>str</code>	字符串	<code>let s: str = "hi";</code>

<code>bigint</code>	任意精度整数	<code>let b: bigint = 999b;</code>
<code>decimal</code>	高精度小数	<code>let d: decimal = 3.14d;</code>
<code>list<T></code>	泛型列表	<code>list<int> = [1, 2]</code>
<code>dict<K, V></code>	字典	<code>dict<str, int></code>
<code>tuple</code>	元组	<code>(1, 2, 3)</code>
<code>channel<T></code>	通道	<code>channel<int></code>
<code>Future<T></code>	冷协程 Future	<code>let f: Future<int> = async_fn();</code>
<code>Task<T></code>	已启动任务句柄	<code>let t: Task<int> = spawn work();</code>
<code>func(T...) → R</code>	函数类型	<code>func(int) → int</code>
<code>dynamic</code>	动态类型	运行时推导

12.2 dynamic

`dynamic` 适合边界数据、快速原型和异构值：

```
let x: dynamic = 42;
print(x);
```

BOLIDE

注意

不要把 `dynamic` 当作默认类型。核心业务数据应尽量使用静态类型，把 `dynamic` 限制在系统边界。

12.3 函数签名类型

```
let f: func(int, int) → int = add;
```

BOLIDE

函数值流转时，显式签名能让错误更早暴露，也能帮助代码阅读。

本章小结

- 局部变量多可推断；公共边界与复杂类型显式标注。
- `dynamic` 是边界工具，不是默认选项。
- 显式函数签名让类型错误更早暴露。

CHAPTER 13

内存管理：ARC、生命周期、weak、unowned

学习目标

- 理解 ARC 引用计数如何自动管理对象生命周期
- 区分 `owned` 与 `ref` 的所有权语义
- 用 `from` 生命周期注解、`weak` / `unowned` 处理引用关系

Bolide 使用 ARC 引用计数管理对象生命周期。普通对象在最后一个强引用离开作用域后释放。

多数代码不需要手动管理内存：

```
class Node {
    name: str;
}

let n = Node("root");
print(n.name);
```

BOLIDE

13.1 owned

`owned` 表示把值的所有权交给函数：

```
fn consume(owned xs: list<int>) {
    print(xs);
}
```

BOLIDE

调用后原变量不再拥有该值。

13.2 ref

`ref` 允许函数修改调用方变量：

```
fn inc(ref n: int) {
    n += 1;
}

let x = 1;
inc(x);
print(x); // 2
```

BOLIDE

13.3 生命周期 from

返回值依赖参数生命周期时，用 `from` 注解表达，可跳过 ARC 开销（借用而非增加引用计数）：

```
fn get_value(ref x: bigint) → bigint from x {
    return x;
}

let a: bigint = 100B;
let b: bigint = get_value(a); // b 借用 a，不增加引用计数
```

编译器对借用施加检查，违反时**编译报错**：返回值必须来源于声明的参数；来源离开作用域后借用不可继续存活（悬空检测）；借用存活期间禁止对来源变量重新赋值；借用值**不允许逃逸**（不能存入容器、不能经通道/`spawn` 跨线程、不能从未声明 `from` 的函数返回）。

让借用逃逸

确实需要把借用值存起来时，显式拷贝后再存储，例如 `bigint(b)`。

13.4 weak 与 unowned

弱引用用 `weak Type` / `unowned Type` 声明（注意不是 `weak<T>`），都不增加引用计数：

```
let obj: Node = Node(42);

let w: weak Node = obj; // 不增加引用计数
print(w.value);         // 访问前自动检查对象是否存活

let u: unowned Node = obj; // 语义上假设对象始终存在
print(u.value);
```

对象被释放后再访问 `weak` / `unowned`，会触发**确定性的运行时错误并中止程序**（带诊断），而不是未定义行为：

```
runtime error: weak/unowned reference accessed after object was deallocated
```

表 13-1 引用方式选择

方式	延长生命周期	语义意图
强引用（默认）	是	绝大多数情况
<code>weak Type</code>	否	对象可能先于引用消亡（如子→父回指）
<code>unowned Type</code>	否	对象保证活得比引用久

13.5 并发安全

- 所有引用计数（强 / 弱）均为**原子操作**（与 Swift / Rust Arc 相同内存序），跨线程 retain/release 无数据竞争。
- `spawn` 对传入对象采用 **move 语义**，原变量失效（运行时 MOVED 标记）。
- channel 当前只支持**值类型**传递。

实践建议

- 默认使用普通强引用。
- 只有为避免引用环时使用 `weak`。
- `unowned` 只用于明确的父子生命周期关系。
- 复杂对象图先画所有权图，再编码。

本章小结

- ARC 自动释放对象，引用计数为原子操作，跨线程安全。
- `owned` 转移所有权、`ref` 原地修改、`from` 借用并约束不可逃逸。
- 弱引用语法 `weak Type` / `unowned Type`，访问已释放对象会确定性 trap。

CHAPTER 14

并发、线程、通道与 async/await

学习目标

- 用 `spawn` / `await` 启动并等待任务
- 理解冷 `Future<T>` 与热 `Task<T>` 的区别
- 用 `spawn all` / `spawn select` / `pool` 组织结构化并发
- 用通道方法与 `select` 在任务间传递消息

14.1 spawn 与 await

`spawn` 启动热任务，返回 `Task<T>`；等待结果统一使用 `await`：

```
fn heavy_work(id: int) → int {
    return id * id;
}

let t: Task<int> = spawn heavy_work(10);
let result: int = await t;
print(result);

let blocking: Task<int> = spawn thread heavy_work(3);
print(await blocking);
```

BOLIDE

14.2 async 函数与冷 Future

`async fn` 调用会创建冷 `Future`——它不会自动并行执行，`await` 时才驱动；`await` 一次只等待一个 `Future`：

```
async fn fetch_data(id: int) → int {
    return id * 10;
}

let f1: Future<int> = fetch_data(1); // 冷 Future, 未自动并行
let f2: Future<int> = fetch_data(2);

let r1: int = await f1;
let r2: int = await f2;
```

BOLIDE

14.3 spawn all（并行等待）

`spawn all` 并发执行一组任务并等待全部完成，结果以元组返回（类型可省略，编译器自动推断）：

```
fn fetch_a() → int { return 100; }
fn fetch_b() → int { return 200; }

let results: (int, int) = spawn all {
    fetch_a(),
    fetch_b()
};
print(results[0]); // 100
print(results[1]); // 200
```

BOLIDE

14.4 spawn select（竞态等待）

`spawn select` 并行启动所有任务，等待第一个完成的任务并执行对应分支：

```
spawn select {
    res1 = task_fast() ⇒ {
        print("fast finished");
    }
    res2 = task_slow() ⇒ {
        print("slow finished");
    }
}
```

BOLIDE

14.5 线程池

```
pool(4) {
    let t1: Task<int> = spawn work(1);
    let t2: Task<int> = spawn work(2);
    print(await t1);
    print(await t2);
}
// pool 块结束时自动等待所有任务完成
```

BOLIDE

14.6 通道

```
let ch: channel<int> = channel();

fn sender(c: channel<int>) → int {
    c.send(42);
    return 0;
}

let t: Task<int> = spawn sender(ch);
let value: int = ch.recv();
print(value);
await t;
```

BOLIDE

14.7 select（通道多路复用）

```
select {
    x = ch1.recv() ⇒ {
        print(x);
    }
    y = ch2.recv() ⇒ {
        print(y);
    }
    timeout(1000) ⇒ {
        print("timeout");
    }
    default ⇒ {
        print("nothing ready");
    }
}
```

BOLIDE

并发实践

- 共享可变状态越少越好。
- 优先用通道传递消息。
- `spawn` 传入对象时注意所有权语义。
- 任务边界要明确处理错误。

本章小结

- `spawn` 返回热 `Task<T>`，用 `await` 取结果。
- `async fn` 产生冷 `Future<T>`；`spawn all` 并行收束返回元组、`spawn select` 取最先完成者。
- 线程池用 `pool(n)`；通道用 `send / recv` 收发，`select` 多路复用。

CHAPTER 15

FFI 与 AOT 发布

学习目标

- 用 `extern` 声明并调用 C 函数
- 用 `export fn` 把 Bolide 函数导出给 C
- 编译静态库 / 可执行文件，并掌握发布前检查项

Bolide 支持双向 C 互操作：既能调用 C 库，也能被 C 程序调用。

15.1 调用 C 函数

源码只写跨平台的逻辑库名，不写 `.dll` / `.so` / `.dylib`。开发期用动态加载 `dyn:`：

```
extern "dyn:c" {
  fn abs(x: c_int) → c_int;
}

extern "dyn:m" {
  fn sqrt(x: c_double) → c_double;
}

let a: int = abs(-42);    // 42
let b: float = sqrt(16.0); // 4.0
```

BOLIDE

也支持把 Bolide 函数作为回调传给 C：

```
fn my_callback(a: int, b: int) → int {
  return a + b;
}

let r: int = test_callback(my_callback, 10, 20);
```

BOLIDE

15.2 外部库标识

`extern "..."` 的库名使用跨平台标识，避免把平台文件名写进源码：

表 15-1 外部库标识

标识	用途	AOT	JIT
<code>bolide</code>	runtime 内置函数	直接链接	直接链接

<code>lib:name</code>	静态库 / 导入库链接	支持	不支持
<code>dyn:name</code>	运行时动态加载	支持	支持
<code>auto:name</code>	JIT 动态、AOT 链接	支持	支持

常用别名 `dyn:c` / `dyn:m`、`lib:c` / `lib:m`、`auto:c` / `auto:m` 分别对应 C 标准库 / 数学库。

关键约束

- `lib:name` 是 AOT-only: JIT 没有链接阶段, 开发期动态加载请用 `dyn:name`。
- raw `extern` 绑定才用 `c_int` / `c_double` / `*char` / `*void` 等 C ABI 类型; 普通 Bolide API 用 `int` / `float` / `str`。
- `int` 是 Bolide 64 位整数, 不等同于 C `int`; 精确匹配 C 签名时用 `c_*` 类型。

15.3 export fn (C 调 Bolide)

用 `export fn` 标记要暴露给 C 的函数 (裸符号名, 无 name mangling), 未标记的函数保持内部命名空间隔离:

```
export fn add(a: int, b: int) → int {
    return a + b;
}
```

BOLIDE

编译静态库并生成头文件:

```
bolide compile mathlib.bl --lib --header
```

BASH

产物:

- Windows: `mathlib.lib`、`mathlib.h`
- Linux/macOS: `libmathlib.a`、`mathlib.h`

C 端链接 Bolide 库 + runtime 静态库即可调用:

```
cl main.c mathlib.lib bolide_runtime.lib # Windows (MSVC)
cc main.c libmathlib.a libbolide_runtime.a # Linux
```

BASH

C 互调 ABI 约定

跨 C 边界仅保证数值与指针签名稳定: `int` / `bool` → `long long`, `float` → `double`; 其余复合类型 (`str` / `list` / 对象) 按运行时内部指针 (`void*`) 传递, C 端无法安全构造。需要 C 友好的导出函数时, 请使用纯数值 / 指针签名。

15.4 AOT 发布

```
bolide compile src/main.bl -o app
```

BASH

发布前检查

- 是否依赖本地相对路径文件。
- 是否需要动态库。
- 是否需要 runtime 静态库参与链接。
- 是否在目标平台验证过。

本章小结

- `extern` 区分 `dyn:`（开发）与 `lib:`（链接）两种来源。
- `export fn` + `--lib --header` 把 Bolide 当作 C 库提供。
- AOT 发布要核对路径依赖、动态库、runtime 链接与目标平台。

CHAPTER 16

Web、模板、数据库与 GUI 标准库

学习目标

- 用 Web 标准库搭建路由与响应
- 用模板生成 HTML、用数据库标准库做基本 CRUD
- 用 GUI 标准库声明式绘制界面

16.1 Web

Web 标准库目标是用简洁 API 写出接近 FastAPI 体验、又能 AOT 编译为原生单文件的服务。请求处理函数接收 `web.Request`、返回 `web.Response`：

```
import "std/web/web.bl";

fn index(req: web.Request) → web.Response {
    return web.html("<h1>Hello Bolide</h1><p>path=" + req.path() + "</p>");
}

fn hello(req: web.Request) → web.Response {
    return web.text("hello " + req.path_param("name"));
}

let app: web.App = web.app();
app.get("/", index);
app.get_async("/hello/{name}", hello);
app.static_files("/static", "public");

app.run("127.0.0.1", 8080);
```

表 16-1 Web 标准库能力

方面	支持
HTTP 方法	GET / POST / PUT / PATCH / DELETE / HEAD / OPTIONS / TRACE / CONNECT
路由	精确路径、 <code>/posts/{id}</code> 动态参数、静态文件目录
请求读取	method / path / query / header / cookie / query 参数 / form 参数 / path 参数 / body 文本与 bytes
响应构造	<code>text</code> / <code>html</code> / <code>json</code> / <code>bytes</code> / <code>empty</code> / <code>redirect</code> ，自定义 status/header/cookie

会话 session id、get/set/contains/remove/clear/destroy/regenerate

16.2 模板

模板引擎用 `template.render(模板, 数据字典)` 渲染；数据为 `dict<str, dynamic>`：

```
import "std/template/template.bl";

let html: str = template.render(
    "<h1>{{ title }}</h1>{% for post in posts %}<article>{{ post.title }}</article>{% endfor %}",
    {
        "title": "Blog",
        "posts": posts,
    }
);
print(html);
```

表 16-2 模板语法

语法	含义
<code>{{ expr }}</code>	输出并 HTML 转义 （适合页面文本）
<code>{!! expr !!}</code>	输出 原始 HTML （仅用于可信内容）
<code>{% if cond %}...{% else %}...{% endif %}</code>	条件渲染
<code>{% for x in xs %}...{% endfor %}</code>	遍历列表
<code>post.title</code>	点路径读取字典 / 对象字段

16.3 数据库

文件数据库以目录为存储位置，表按文件保存。用户侧只接触 `str`、`dict<str, dynamic>`、`list<dict<str, dynamic>>` 和 `Database` 等 Bolide 类型：

BOLIDE

```
import "std/db/db.bl";

let database: db.Database = db.open("data/blog");
database.create_table("posts", "title,slug,body,published");

let row: dict<str, dynamic> = {
  "title": "Hello Bolide",
  "slug": "hello-bolide",
  "body": "A first post.",
  "published": true,
};
database.insert("posts", row);

let posts: list<dict<str, dynamic>> = database.all("posts");
database.close();
```

数据库方法：`create_table`、`insert`、`update`、`delete`、`get`、`all`、`where_eq`、`count`、`last_error`。

16.4 GUI

GUI 标准库后端为 `egui/eframe`，声明式渲染：状态存在全局变量或对象中，`gui.run(title, w, h, root)` 每帧调用 `root(ui)` 绘制界面。`ui.button` 返回 `bool` 表示是否被点击：

```

import "std/gui/gui.bl";

let count: int = 0;
let status: str = "准备";

fn toolbar(ui: gui.Ui) {
    if ui.button("增加") {
        count = count + 1;
        status = "计数 " + str(count);
    }
    if ui.button("清零") {
        count = 0;
        status = "已清零";
    }
}

fn body(ui: gui.Ui) {
    ui.heading("Bolide GUI");
    ui.pack_left(8, toolbar);
    ui.space(12);
}

fn root(ui: gui.Ui) {
    ui.pad(16, 16, body);
}

gui.run("Bolide GUI", 420, 280, root);

```

表 16-3 GUI 控件与布局

类别	API
文本	label / heading / small / strong
命令与选择	button / selectable / link
输入	text_input / password_input / multiline_input / checkbox / slider
状态	progress / separator / space
布局	pad / pack_top·left·right·bottom / row / column / grid / fill / frame / scroll

提示

GUI 与 Web 都建议先用 JIT 快速迭代，发布时用 AOT 编译为单文件（GUI/Web runtime 会静态链接进最终程序）。

本章小结

- Web: `web.app()` + 路由 + `web.text/html/json` 响应 + 会话, 可 AOT 单文件发布。
- 模板支持 `{{ }}` / `{!! !!}` / `{% if %}` / `{% for %}` 与点路径。
- 数据库用 `db.open` + `create_table` / `insert` / `all` 等方法; GUI 用 `gui.run` + 声明式布局。

CHAPTER 17

报错诊断、调试与测试

学习目标

- 读懂 CLI 的源码级报错诊断
- 识别常见错误类型及其处理方式
- 用最小测试覆盖语义边界

Bolide 0.12.3 起，CLI 提供源码级报错诊断。`run`、`compile`、`compile --lib` 和 REPL 都会显示：

- 错误阶段：`bolide::parse` 或 `bolide::compile`
- 文件名、行列号、源码片段、caret 标注、针对性 help

示例：

```
let x = missing_name + 1;
print(x);
```

BOLIDE

输出：

Error: bolide::compile

TEXT

```
× Compile error: Undefined variable or function: missing_name
  └─[example.bl:1:9]
1 | let x = missing_name + 1;
  |               ^
  |               └─ 'missing_name' is not defined
2 | print(x);
  |
  help: Define the name before using it, or check for a spelling/import mistake.
```

常见错误：

表 17-1 常见错误与处理

错误	原因	处理
Undefined variable	名字未定义或拼写错误	检查作用域、导入和拼写
Undefined function	函数不存在或未导入	定义函数或导入模块
Unknown method	接收者类型没有该方法	检查类型和方法名
missing required argument	调用缺少必填参数	补充位置参数或具名参数

Failed to parse module

import 的文件语法错误

直接运行被导入文件查看位置

17.1 测试

一个测试文件可以用打印 **PASS** / **FAIL** 的方式表达：

```
fn expect(label: str, ok: bool) {  
    if ok {  
        print("PASS " + label);  
    } else {  
        print("FAIL " + label);  
    }  
}  
  
expect("add", 1 + 1 == 2);
```

BOLIDE

运行：

```
bolide run tests/test_add.bl
```

BASH

测试建议

- 语义边界写小测试。
- 对函数值、闭包、所有权、异常、并发写回归测试。
- JIT 和 AOT 都要覆盖核心路径。

本章小结

- 报错带阶段、行列、源码片段与 help，定位成本低。
- 记住五类高频错误的根因与处理路径。
- 测试聚焦语义边界，并同时覆盖 JIT 与 AOT。

CHAPTER 18

性能优化与工程实践

学习目标

- 建立「先测量、再优化」的习惯
- 掌握分配、字符串、集合、并发的优化要点
- 形成可维护的工程实践约定

性能优化先测量，再修改。

18.1 优先级

1. 减少不必要的分配。
2. 避免在热路径使用 `dynamic`。
3. 大集合处理时减少中间列表。
4. 频繁调用的函数写清楚类型签名。
5. 发布时使用 AOT。

18.2 字符串

大量拼接时，尽量把片段组织好，避免循环里反复创建大字符串。

```
let out = "";
for item in items {
    out = out + render_item(item);
}
```

BOLIDE

如果性能敏感，应改成批量渲染或模板。

18.3 集合

`map` / `filter` 清晰，但在超热路径中可能产生中间集合。必要时使用显式循环。

18.4 并发

线程不是越多越快。对于 CPU 密集任务，线程池大小通常接近核心数；对于 IO 密集任务，可以更高，但要用压测确认。

18.5 工程实践

- 公共边界显式写类型。
- 错误类型用类或 enum 建模。
- 模块之间只暴露必要函数。
- 测试覆盖语义边界，不只覆盖 happy path。
- 发布前用 `bolide compile` 验证 AOT 路径。

本章小结

- 优化顺序：测量 → 减分配 → 避热路径 `dynamic` → 减中间集合 → AOT。
- 字符串与集合在热路径上警惕中间产物。
- 线程数据量按 CPU/IO 性质压测确定，不盲目加大。

CHAPTER 19

综合项目：命令行任务管理器

本章把前面的知识组合成一个小项目，串联类、列表、循环、函数与编译发布。

项目目标

- 添加任务
- 列出任务
- 标记完成

19.1 数据模型

```
class TodoItem {  
    id: int;  
    title: str;  
    done: bool = false;  
}
```

BOLIDE

19.2 基本操作

```
fn add_task(tasks: list<TodoItem>, title: str) → list<TodoItem> {
    let id = tasks.len() + 1;
    tasks.push(TodoItem(id, title, false));
    return tasks;
}

fn list_tasks(tasks: list<TodoItem>) {
    for item in tasks {
        let mark = " ";
        if item.done {
            mark = "x";
        }
        print(str(item.id) + ". [" + mark + "] " + item.title);
    }
}

fn finish(tasks: list<TodoItem>, id: int) → list<TodoItem> {
    for item in tasks {
        if item.id == id {
            item.done = true;
        }
    }
    return tasks;
}
```

BOLIDE

19.3 主程序

```
let tasks: list<TodoItem> = [];

tasks = add_task(tasks, "learn Bolide");
tasks = add_task(tasks, "write a program");
tasks = finish(tasks, 1);

list_tasks(tasks);
```

BOLIDE

运行：

```
bolide run todo.bl
```

BASH

编译：

```
bolide compile todo.bl -o todo
```

BASH

扩展方向

- 用文件保存任务。
- 用数据库标准库存储任务。
- 用 Web 标准库做浏览器界面。
- 用 GUI 标准库做桌面界面。

本章小结

- 用类建模实体、用列表存集合、用函数封装操作。
- 同一份代码可 `run` 调试、可 `compile` 发布。
- 沿数据库 / Web / GUI 方向都能自然扩展。

CHAPTER 20

附录：常用语法速查

20.1 变量

```
let x: int = 1;
let y = 2;
x += 1;
```

BOLIDE

20.2 函数

```
fn add(a: int, b: int) → int {
    return a + b;
}
```

BOLIDE

20.3 默认参数与具名参数

```
fn greet(name: str = "world") {
    print(name);
}

greet(name="Bolide");
```

BOLIDE

20.4 列表

```
let xs: list<int> = [1, 2, 3];
xs.push(4);
print(xs[0]);
```

BOLIDE

20.5 字典

```
let m: dict<str, int> = {"a": 1};
m["b"] = 2;
```

BOLIDE

20.6 类

```
class Point {
    x: int;
    y: int;
}

let p = Point(1, 2);
```

BOLIDE

20.7 闭包

```
let f: func(int) → int = fn(x: int) → int {
    return x + 1;
};
```

BOLIDE

20.8 异常

```
try {
    throw Error("error");
} catch (e: Error) {
    print(e.message);
} finally {
    print("cleanup");
}
```

BOLIDE

20.9 Result / Option

```
fn read_id(raw: str) → Result<int, Error> {
    let id: int = int(raw);
    return Result.Ok(id);
}

fn checked_id(raw: str) → Result<int, Error> {
    let id: int = read_id(raw)?;
    return Result.Ok(id + 1);
}
```

BOLIDE

20.10 并发与通道

```
fn work(x: int) → int {
    return x * 2;
}

async fn fetch(id: int) → int {
    return id * 10;
}

let task: Task<int> = spawn work(21);
let value: int = await task;

let f: Future<int> = fetch(1);
print(await f);

let ch: channel<int> = channel();
ch.send(42);
print(ch.recv());
```

BOLIDE

20.11 AOT 与静态库

```
bolide compile main.bl -o main
```

BASH

```
export fn add(a: int, b: int) → int {
    return a + b;
}
```

BOLIDE

```
bolide compile lib.bl --lib --header
```

BASH

20.12 关键字速览

表 20-1 按用途分类的关键字（基于全书示例）

类别	关键字
绑定 / 函数	let · fn · return · self
控制流	if · elif · else · while · for · in · break · continue · match
类型 / 数据	class · enum · union
错误处理	try · catch · finally · throw · throws
模块	import · as

并发	<code>spawn</code> · <code>await</code> · <code>async</code> · <code>all</code> · <code>pool</code> · <code>select</code> · <code>timeout</code> · <code>default</code>
内存 / 所有权	<code>owned</code> · <code>ref</code> · <code>weak</code> · <code>unowned</code> · <code>from</code>
FFI	<code>extern</code> · <code>export</code>
逻辑 / 字面量	<code>and</code> · <code>or</code> · <code>not</code> · <code>true</code> · <code>false</code> · <code>none</code>

EPILOGUE

结语

学会 Bolide 的关键不是记住所有语法，而是掌握几条主线：

- 用静态类型描述核心数据。
- 用函数和闭包表达可组合逻辑。
- 用类管理有状态对象。
- 用模块和包管理工程边界。
- 用 ARC、所有权和生命周期避免资源错误。
- 用 JIT 快速开发，用 AOT 稳定发布。

读完这本书后，你已经具备从零写出 Bolide 程序、组织中型项目、诊断错误、编写测试、接入 C、发布原生可执行文件的完整路径。

— Bolide 0.12.3 · 从入门到精通 —